For this lab, the goal was to improve our image classifier by moving from a basic neural network to a Convolutional Neural Network (CNN). The main task was to see if this more advanced setup could do a better job of telling the difference between a chihuahua and a muffin.

The way this CNN looks at images is very different from the model we used in the last lab. In the previous workshop, the model treated an image like a long, flat list of numbers, which meant it lost track of where pixels were in relation to each other. This CNN in this lab uses convolutional layers that act more like a scanner. As mentioned in the PyTorch Documentation, these layers scan the image to find patterns like edges, circles, or textures. This helps the model see shapes and objects much more like a human would (Xu, 2025). It also uses pooling layers, which basically simplify the image data so the computer can focus on the most important parts.

When I first ran the lab with the default settings, the CNN got 83.33% accuracy. However, after I played around with the settings, I was able to get the CNN up to 96.67%. This showed me that while CNNs are more powerful, they need a bit more tuning to work right. I major thing I noticed was that the CNN was much slower to train. It takes more math for the computer to scan for those patterns than it does to just read a flat list of numbers, so each round of training took longer than the last lab.

The hardest part was trying to get the accuracy higher than that initial 83%. I followed the lab's advice and changed some of the hyperparameters. My first attempt actually made things worse, I lowered the learning rate to 0.0001 which turned out be too much, and the model learned so slowly that it couldn't finish in time. To fix this, I found a better middle ground with a learning rate of 0.0008 and slightly increased the image rotation settings so the model saw more variety. These small changes worked, and that's how I finally hit 96.67%.

This kind of technology is used for a lot more than just muffins. As I discovered in the previous lab it is used in hospitals to look at X-rays or in self-driving cars to spot people on the street. One issue with this technology is data bias which is if our training photos only show one type of dog, the model won't be fair or accurate when it sees others. As Xu (2025) points out, having a diverse mix of data is a major part of making AI fair. There's also the worry that the same technology could be used for things like unwanted surveillance, so we have to be careful about how these models are deployed.

This lab was really interesting because it showed me how much of a difference tiny changes can make. It was a bit frustrating when my first experiment failed, but it actually helped me understand how the learning process works. It was a great feeling to see the

final results page covered in green labels showing that the model got almost everything right. I feel much more comfortable with these tools now, and I understand why CNNs are the go-to choice for anything involving images.

References

- **PyTorch.** (2025). Adam Optimizer. *PyTorch Documentation*.
- **PyTorch.** (2025). Conv2d. *PyTorch Documentation*.
- **Xu, J.** (2025). Understanding Convolutional Neural Networks for Image Classification. *Transactions on Computer Science and Intelligent Systems Research*.